

Fase 5 - Lombok

Esse é o último material da trilha. Se você fez as fases anteriores, já escreveu getters, setters e construtores na mão várias vezes — o Lombok existe justamente pra eliminar essa repetição. Por isso ele vem por último: agora você sabe exatamente o que está por trás de cada anotação dele, em vez de decorar sem entender o que está sendo economizado.

1. O que é o Lombok

Lombok é uma biblioteca que gera código repetitivo (boilerplate) automaticamente durante a compilação, a partir de anotações que você coloca na classe. Ele não faz “mágica em tempo de execução” — o código dos getters, setters, construtores etc. é gerado de verdade, só que você não precisa escrever nem ver ele.

Antes de usar, é preciso adicionar a dependência no `pom.xml` (revise a Fase 2 se precisar relembrar como funciona):

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

No IntelliJ, também é preciso instalar o plugin “Lombok” (Settings > Plugins) pra IDE entender e sugerir os métodos gerados — sem o plugin, o código roda, mas o IntelliJ vai sublinhar tudo de vermelho achando que os métodos não existem.

2. @Getter e @Setter

Lembra da classe `Produto` da Fase 3, onde você escrevia isso na mão?

```
public class Produto {
    private Long id;
    private String nome;
    private double preco;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getName() { return nome; }
    public void setName(String nome) { this.nome = nome; }
    public double getPreco() { return preco; }
    public void setPreco(double preco) { this.preco = preco; }
}
```

Com Lombok, isso vira:

```
@Getter
@Setter
public class Produto {
    private Long id;
    private String nome;
    private double preco;
}
```

@Getter e @Setter podem ser colocados na classe inteira (gera pra todos os campos) ou em um campo específico, se você quiser um getter só em alguns atributos:

```
public class Produto {
    @Getter @Setter
    private String nome;

    @Getter // sem setter - esse campo so pode ser lido de fora, nao alterado
    private Long id;
}
```

3. Construtores: @NoArgsConstructor, @AllArgsConstructor, @RequiredArgsConstructor

- **@NoArgsConstructor**: gera um construtor vazio, sem parâmetros.
- **@AllArgsConstructor**: gera um construtor com todos os campos da classe, na ordem em que foram declarados.
- **@RequiredArgsConstructor**: gera um construtor só com os campos que são final (ou marcados com @NonNull).

```
@NoArgsConstructor
@AllArgsConstructor
public class Produto {
    private Long id;
    private String nome;
    private double preco;
}
// Gera: Produto() {}
// Gera: Produto(Long id, String nome, double preco) { ... }
```

Lembra da Fase 4, onde vimos que injeção de dependência por construtor é a forma recomendada?

```
@Service
public class ProdutoService {
    private final ProdutoRepository repository;

    public ProdutoService(ProdutoRepository repository) {
        this.repository = repository;
    }
}
```

Com `@RequiredArgsConstructor`, esse construtor manual desaparece:

```
@Service
@RequiredArgsConstructor
public class ProdutoService {
    private final ProdutoRepository repository;
}
```

O Lombok gera o construtor com o campo `final`, e — como você viu na Fase 4 — desde o Spring 4.3, um único construtor já é injetado automaticamente, sem precisar de `@Autowired` em lugar nenhum. É por isso que esse combo (`@RequiredArgsConstructor` + campos `final`) virou o padrão em projetos Spring modernos.

4. @Data

`@Data` é uma anotação “combo”: sozinha, ela já ativa várias outras de uma vez.

```
@Data
public class Produto {
    private Long id;
    private String nome;
    private double preco;
}
```

Isso equivale a colocar, tudo junto: `@Getter`, `@Setter`, `@ToString`, `@EqualsAndHashCode` e `@RequiredArgsConstructor`.

Cuidado com `@Data` em entidades JPA (classes com `@Entity`, que você viu na Fase 4): o `equals/hashCode` gerado usa todos os campos da classe. Se a entidade tiver um relacionamento com outra entidade (uma lista de itens, por exemplo), isso pode causar loop infinito ou erro de carregamento (*lazy loading*). Por esse motivo, muitos times evitam `@Data` em entidades e preferem `@Getter/@Setter` separados, com `@EqualsAndHashCode` configurado manualmente pra considerar só o `id`.

```
@Entity
@Getter
@Setter
@EqualsAndHashCode(of = "id") // considera só o id na comparacao
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nome;
    private double preco;
}
```

5. @Builder

`@Builder` gera um jeito alternativo de criar objetos, encadeando a definição de cada campo — útil quando a classe tem muitos atributos e você quer deixar claro, na hora

de criar o objeto, qual valor é qual (em vez de um construtor com uma lista grande de parâmetros posicionais).

```
@Builder
public class Produto {
    private Long id;
    private String nome;
    private double preco;
}
```

Uso:

```
Produto produto = Produto.builder()
    .id(1L)
    .nome("Mouse")
    .preco(59.90)
    .build();
```

Compare com o construtor tradicional gerado por `@AllArgsConstructor`:

```
Produto produto = new Produto(1L, "Mouse", 59.90);
```

Com poucos campos, a diferença é pequena. Mas em classes com 6, 8, 10 atributos, o `builder()` deixa muito mais claro (e menos propenso a erro) qual valor está indo pra qual campo.

6. @Slf4j

Essa anotação gera automaticamente um objeto de log dentro da classe, pronto pra usar:

```
@Slf4j
@Service
public class ProdutoService {

    public void cadastrar(Produto produto) {
        log.info("Cadastrando produto: {}", produto.getNome());
        // ...
    }
}
```

Sem o Lombok, você precisaria escrever isso manualmente em toda classe que quisesse logar algo:

```
private static final Logger log = LoggerFactory.getLogger(ProdutoService.class);
```

`@Slf4j` só economiza essa linha repetitiva — o `log.info(...)`, `log.error(...)` etc. funcionam exatamente como funcionariam sem o Lombok.

7. Resumo de quando usar cada uma

- **DTOs e classes simples** (sem relacionamento com outras entidades): `@Data` é seguro e prático.

- **Entidades JPA** (@Entity): prefira @Getter/@Setter separados, e configure @EqualsAndHashCode manualmente pelo id.
- **Services, Controllers, Repositories customizados**: @RequiredArgsConstructor com campos final pra injeção de dependência.
- **Classes com muitos atributos opcionais**: @Builder facilita a criação de objetos legível.
- **Qualquer classe que precise logar algo**: @Slf4j.

Exercício 5.1 — Refatorar com Lombok Pegue a classe Cliente que você criou na Fase 4 (a entidade com @Entity, @Id, @GeneratedValue) e substitua os getters/setters escritos na mão pelas anotações do Lombok, seguindo a recomendação da seção 4 (não use @Data direto na entidade — use @Getter/@Setter/@EqualsAndHashCode(of = "id")).

Exercício 5.2 — Injeção de dependência com Lombok No ClienteService e no ClienteController da Fase 4, troque os construtores escritos manualmente por @RequiredArgsConstructor, garantindo que os campos continuem final. Rode a aplicação de novo e confirme que a injeção de dependência continua funcionando normalmente.

Exercício 5.3 — Builder Crie uma classe Pedido com pelo menos 5 atributos (por exemplo: id, cliente, dataCriacao, valorTotal, status). Anote ela com @Builder e @Getter. No main, crie um objeto Pedido usando o builder, encadeando cada campo. Depois, escreva em 2-3 linhas o que você achou mais legível: o builder ou um construtor tradicional com todos os parâmetros — e por quê.

Exercício 5.4 — Investigando o cuidado do @Data Crie duas entidades JPA relacionadas: Pedido (com uma lista de ItemPedido) e ItemPedido (com uma referência de volta pro Pedido, tipo @ManyToOne). Anote as duas com @Data. Gere o toString() de um Pedido que tenha itens e observe o que acontece no console (pesquise sobre “loop infinito toString JPA bidirecional” se precisar de uma pista). Depois, troque @Data por @Getter/@Setter nas duas classes e confirme que o problema desaparece. Escreva um resumo curto do que você entendeu sobre esse cuidado.

Fechando a trilha

Com essas cinco fases, você passou por: fundamentos de orientação a objetos, gerenciamento de dependências com Maven, separação em camadas (MVC), como o Spring Boot automatiza essa separação, e como o Lombok reduz o código repetitivo de tudo isso. A partir daqui, o próximo passo natural é praticar construindo uma API um pouco maior, unindo tudo — várias entidades relacionadas, regras de negócio mais elaboradas nos Services, e endpoints REST completos — pra consolidar o que foi visto em cada fase de forma integrada.